

# Proyecto: Reconocimiento Facial

Francisco Romero Mestiza

19 de marzo de 2026

## Índice

|                                  |          |
|----------------------------------|----------|
| <b>1. Enlaces</b>                | <b>2</b> |
| <b>2. Introducción</b>           | <b>2</b> |
| <b>3. Objetivo</b>               | <b>2</b> |
| <b>4. Metodología</b>            | <b>3</b> |
| 4.1. Pre-entrenamiento . . . . . | 3        |
| 4.2. Entrenamiento . . . . .     | 3        |
| 4.3. Transfer learning . . . . . | 5        |
| <b>5. Aumento de datos</b>       | <b>5</b> |
| 5.1. Fine Tunning . . . . .      | 7        |
| <b>6. Prueba del modelo</b>      | <b>8</b> |

## 1. Enlaces

(a) Entrenar la red convolucional con CelebA

Colab:

[https://colab.research.google.com/drive/1hA8\\_Y9z2l0hQJbInGIbK5u-e9RRdGZM4?usp=sharing](https://colab.research.google.com/drive/1hA8_Y9z2l0hQJbInGIbK5u-e9RRdGZM4?usp=sharing)

Dagshub.

[https://dagshub.com/CrUz-035/Tarea\\_4.mlflow/#/experiments/0/runs?searchFilter=&orderByKey=attributes.start\\_time&orderByAsc=false&startTime=ALL&lifecycleFilter=Active&modelVersionFilter=All+Runs&datasetsFilter=W10%3D](https://dagshub.com/CrUz-035/Tarea_4.mlflow/#/experiments/0/runs?searchFilter=&orderByKey=attributes.start_time&orderByAsc=false&startTime=ALL&lifecycleFilter=Active&modelVersionFilter=All+Runs&datasetsFilter=W10%3D)

(b) Entrenar el clasificador usando las capas convolucionales de la red pre-entrenada

(c) Entrenar las últimas capas convolucionales y densas de la red para un refinamiento.

Colab:

<https://colab.research.google.com/drive/1cYi52B89k-yo13cvweHHPLuwL3GYjqag?usp=sharing>

Dagshub:

Transfer Learning:

[https://dagshub.com/CrUz-035/Preentrenado.mlflow/#/experiments/0/runs?searchFilter=&orderByKey=attributes.start\\_time&orderByAsc=false&startTime=ALL&lifecycleFilter=Active&modelVersionFilter=All+Runs&datasetsFilter=W10%3D](https://dagshub.com/CrUz-035/Preentrenado.mlflow/#/experiments/0/runs?searchFilter=&orderByKey=attributes.start_time&orderByAsc=false&startTime=ALL&lifecycleFilter=Active&modelVersionFilter=All+Runs&datasetsFilter=W10%3D)

Fine Tunning:

[https://dagshub.com/CrUz-035/Fine\\_tunning.mlflow/#/experiments/1/runs?searchFilter=&orderByKey=attributes.start\\_time&orderByAsc=false&startTime=ALL&lifecycleFilter=Active&modelVersionFilter=All+Runs&datasetsFilter=W10%3D](https://dagshub.com/CrUz-035/Fine_tunning.mlflow/#/experiments/1/runs?searchFilter=&orderByKey=attributes.start_time&orderByAsc=false&startTime=ALL&lifecycleFilter=Active&modelVersionFilter=All+Runs&datasetsFilter=W10%3D)

Prueba del programa:

<https://colab.research.google.com/drive/19nLu8qABLM67iwVwTeKP6EqAt4NL3fk?usp=sharing>

## 2. Introducción

El proyecto tiene como propósito el uso de la librería *CelebA*, así como la aplicación de técnicas de *Transfer Learning* y *Fine Tuning* para entrenar una red neuronal capaz de predecir si el rostro de una persona en una imagen corresponde a un usuario específico.

## 3. Objetivo

Desarrollar un modelo que sea capaz de distinguir si una imagen corresponde al rostro del usuario o no, utilizando técnicas de aprendizaje profundo.

## 4. Metodología

### 4.1. Pre-entrenamiento

Para el pre-entrenamiento del modelo se utiliza la base de datos CelebA, la cual contiene miles de imágenes de rostros con etiquetas asociadas a distintos atributos faciales. Asimismo, se emplea una base de datos propia del usuario.

Para acceder a la base de datos de CelebA, se inicia sesión en Kaggle, posteriormente se vincula con Google Colab y se adjunta el dataset correspondiente.

El modelo final se construyó:

- Reutilizando las capas convolucionales del modelo pre-entrenado
- Eliminando el clasificador original
- Añadiendo nuevas capas densas para clasificación binaria

### 4.2. Entrenamiento

El entrenamiento se realizó en tres etapas:

1. Entrenamiento con CelebA
2. Entrenamiento del clasificador
3. Fine-tuning de las últimas capas

#### **Pre-entrenamiento del modelo**

Se entrena una red neuronal convolucional utilizando la base de datos CelebA para predecir atributos faciales. Se generaron subconjuntos de entrenamiento y validación, considerando un total de 50,000 imágenes, de las cuales 35,000 se utilizaron para entrenamiento y 15,000 para validación.

Posteriormente, se define la arquitectura del modelo, la cual se muestra en la Figura 11. Este modelo constituye la base para las etapas posteriores.

Las gráficas de desempeño del modelo durante el entrenamiento se presentan a continuación.

Parámetros:

- Épocas: 40
- Learning rate: 0.001
- Función de activación: ReLU
- Optimizador: Adam
- Función de activación de salida: Sigmoid
- Accuracy: 0.90

```

8 inputs = keras.Input(shape=(150, 150, 3))
9 x = inputs
10
11 def block(x, base_filter=32, pooling=True):
12     #Construye bloques de datos que se pueden volver a utilizar.
13     residual = x #Se conserva dato para ocuparlo más tarde.
14     #Se aplica convolucional
15     x = layers.Conv2D(base_filter, 3, activation="relu", padding="same")(x)
16     #Capa convolucional con el doble de filtros
17     x = layers.Conv2D(2*base_filter, 3, activation="relu", padding="same")(x)
18     if (pooling == True): #Se reducen dimensiones espaciales.
19         x = layers.MaxPooling2D(2, padding="same")(x)
20         residual = layers.Conv2D(2*base_filter, 1, strides=2)(residual)
21     else:
22         #Debe coincidir la dimensión con x.
23         residual = layers.Conv2D(2*base_filter, 1)(residual)
24     x = layers.add([x, residual]) #Se suma x original.
25     return x
26
27 x = block(x, base_filter=32)
28 x = block(x, base_filter=48)
29 x = block(x, base_filter=64, pooling=False)
30
31 x = layers.GlobalAveragePooling2D()(x)
32 #Se reduce mapa de características a un promedio
33 #Drop Out
34 x = layers.Dropout(0.3)(x)
35
36 #Sigmoide para salida de datos, como son 40 clasificaciones, se especifica.
37 outputs = layers.Dense(40, activation="sigmoid")(x)
38 model = keras.Model(inputs=inputs, outputs=outputs)

```

Figura 1: Modelo base

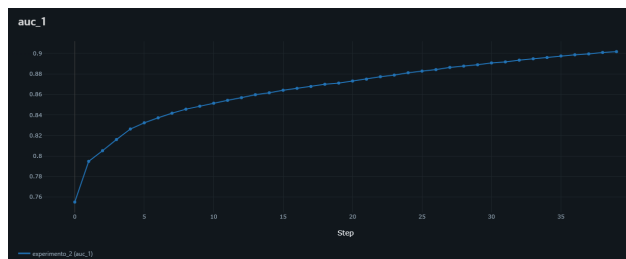


Figura 2: Evolución de la accuracy durante el entrenamiento

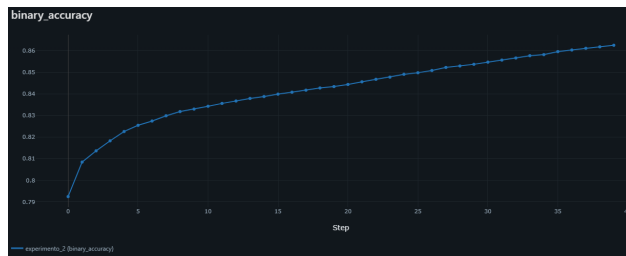


Figura 3: Evolución de la binary accuracy

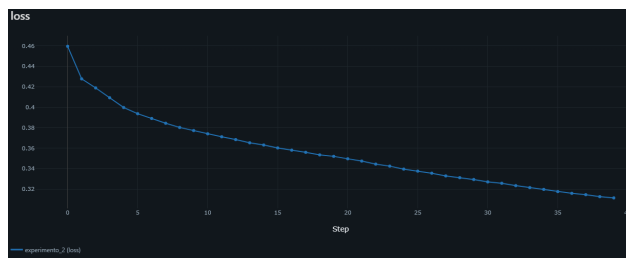


Figura 4: Evolución de la función de pérdida

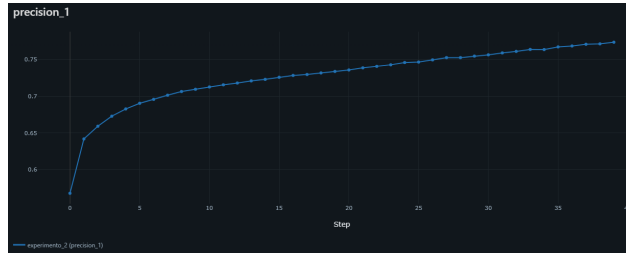


Figura 5: Precisión del modelo durante el entrenamiento

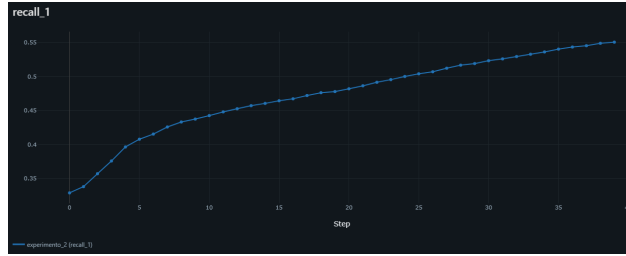


Figura 6: Recall del modelo durante el entrenamiento

- Recall: 0.55
- Binary Accuracy: 0.86
- Loss: 0.31

Los resultados obtenidos indican un buen desempeño general del modelo en términos de accuracy. Sin embargo, el valor de recall sugiere que el modelo presenta dificultades para identificar correctamente todos los casos positivos. Este experimento se encuentra en el enlace a Dagshub al inicio del documento, el último experimento.

### 4.3. Transfer learning

Para esta fase, se utilizó el modelo previamente entrenado, congelando sus pesos y reemplazando la última capa por un clasificador binario. Esto permite adaptar las características aprendidas en el pre-entrenamiento para una tarea específica.

## 5. Aumento de datos

Para la base de datos de fotos propias, se utilizó la función *ImageDataGenerator*. Con el objetivo de mejorar la capacidad de generalización del modelo, se aplicaron transformaciones como:

- Rotación
- Zoom
- Volteo horizontal

| Model: "functional_3"                            |                      |         |                                    |
|--------------------------------------------------|----------------------|---------|------------------------------------|
| Layer (type)                                     | Output Shape         | Param # | Connected to                       |
| input_layer_3 (InputLayer)                       | (None, 150, 150, 1)  | 0       | -                                  |
| conv2d_18 (Conv2D)                               | (None, 150, 150, 32) | 896     | input_layer_3[0]_                  |
| conv2d_19 (Conv2D)                               | (None, 150, 150, 64) | 18,496  | conv2d_18[0][0]                    |
| max_pooling2d_4 (MaxPooling2D)                   | (None, 75, 75, 64)   | 0       | conv2d_19[0][0]                    |
| conv2d_20 (Conv2D)                               | (None, 75, 75, 64)   | 256     | input_layer_3[0]_                  |
| add_6 (Add)                                      | (None, 75, 75, 64)   | 0       | max_pooling2d_4[0]_conv2d_20[0][0] |
| conv2d_21 (Conv2D)                               | (None, 75, 75, 48)   | 27,696  | add_6[0][0]                        |
| conv2d_22 (Conv2D)                               | (None, 75, 75, 96)   | 41,568  | conv2d_21[0][0]                    |
| max_pooling2d_5 (MaxPooling2D)                   | (None, 38, 38, 96)   | 0       | conv2d_22[0][0]                    |
| conv2d_23 (Conv2D)                               | (None, 38, 38, 96)   | 6,240   | add_6[0][0]                        |
| add_7 (Add)                                      | (None, 38, 38, 96)   | 0       | max_pooling2d_5[0]_conv2d_23[0][0] |
| conv2d_24 (Conv2D)                               | (None, 38, 38, 64)   | 55,360  | add_7[0][0]                        |
| conv2d_25 (Conv2D)                               | (None, 38, 38, 128)  | 73,856  | conv2d_24[0][0]                    |
| conv2d_26 (Conv2D)                               | (None, 38, 38, 128)  | 12,416  | add_7[0][0]                        |
| add_8 (Add)                                      | (None, 38, 38, 128)  | 0       | conv2d_25[0][0], conv2d_26[0][0]   |
| global_average_pooling2d_1 (GlobalAveragePool2D) | (None, 128)          | 0       | add_8[0][0]                        |
| dropout_2 (Dropout)                              | (None, 128)          | 0       | global_average_pooling2d_1[0]      |
| dense_6 (Dense)                                  | (None, 256)          | 33,024  | dropout_2[0][0]                    |
| dropout_3 (Dropout)                              | (None, 256)          | 0       | dense_6[0][0]                      |
| dense_7 (Dense)                                  | (None, 1)            | 257     | dropout_3[0][0]                    |

Figura 7: Modelo - Transfer Learning

Esta función se puede observar en los enlaces a google colab al inicio de este documento. La cantidad inicial de fotos propias es alrededor de 1600.

El modelo utilizado en esta fase se muestra en la Figura 7.

Las siguientes gráficas muestran el comportamiento del modelo durante el entrenamiento con aumento de datos. Este entrenamiento se encuentra en el enlace a Dagshub al inicio del documento, el último experimento.

Parámetros:

- **Épocas:** 6
- **Optimizador:** RMSprop
- **Learning Rate (lr):** 0.001
- **Función de activación de salida:** Sigmoid
- **Accuracy:** 0.93

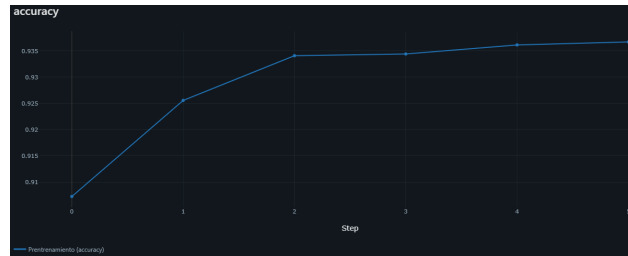


Figura 8: Accuracy durante el entrenamiento con aumento de datos

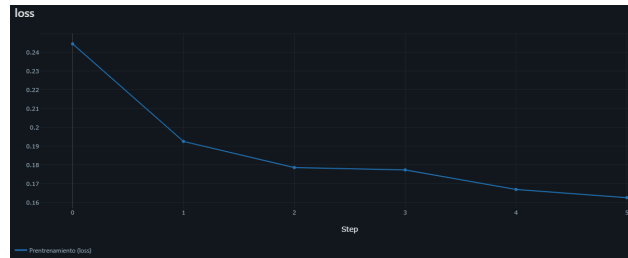


Figura 9: Loss durante el entrenamiento con aumento de datos

- Loss: 0.16

## 5.1. Fine Tunning

Antes de la evaluación final, se descongelaron las últimas capas del modelo y se realizó un nuevo entrenamiento para refinar los parámetros. Aquí se muestran dos gráficas asociadas al entrenamiento. Las demás gráficas se encuentran en el enlace a Dagshub al inicio de este documento, el último experimento.

Parámetros nuevos:

- **Épocas:** 6
- Learning rate: 0.0000001
- Accuracy: 0.9421
- Loss: 0.1617

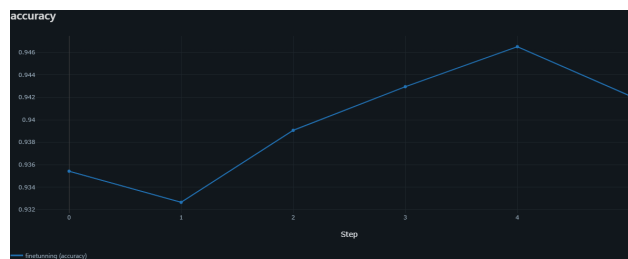


Figura 10: Accuracy durante el fine tuning

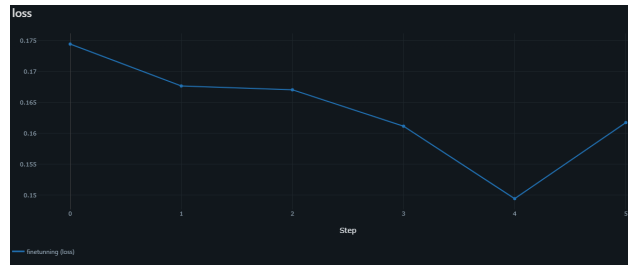


Figura 11: Loss durante el fine tuning

## 6. Prueba del modelo

El modelo se probó con 5 imágenes propias que no formaban parte del conjunto de entrenamiento, así como 5 imágenes de personas distintas.

Valor de predicción (Imágenes propias):

- 0.9915
- 0.9698
- 0.9513
- 0.9737
- 0.9958

Valor de predicción (Imágenes ajenas):

- 0.0003
- 0.8440
- 0.6721
- 0.0733
- 0.2573

Los resultados muestran que el modelo es capaz de identificar correctamente el rostro del usuario en la mayoría de los casos. No obstante, se observan algunos valores altos en imágenes ajenas, lo cual indica la presencia de falsos positivos. Se deduce que el modelo aún puede mejorarse.